IMPROVING NEURAL PITCH ESTIMATION WITH SWIPE KERNELS

David Marttila, Joshua D. Reiss
Centre for Digital Music, Queen Mary University of London
d.sudholt@qmul.ac.uk, joshua.reiss@qmul.ac.uk

ABSTRACT

Neural networks have become the dominant technique for accurate pitch and periodicity estimation. Although a lot of research has gone into improving network architectures and training paradigms, most approaches operate directly on the raw audio waveform or on general-purpose time-frequency representations. We investigate the use of Sawtooth-Inspired Pitch Estimation (SWIPE) kernels as an audio frontend and find that these hand-crafted, task-specific features can make neural pitch estimators more accurate, robust to noise, and more parameter-efficient. We evaluate supervised and self-supervised state-of-the-art architectures on common datasets and show that the SWIPE audio frontend allows for reducing the network size by an order of magnitude without performance degradation. Additionally, we show that the SWIPE algorithm on its own is much more accurate than commonly reported, outperforming state-of-the-art self-supervised neural pitch estimators.

1. INTRODUCTION

Pitch plays a central role in how humans perceive sound. Consequently, pitch estimation is a fundamental task in many music, speech and audio processing pipelines. While pitch is a psychoacoustic phenomenon, it closely correlates to the signal processing concept of the fundamental frequency f_0. Recent literature commonly uses the term "pitch estimation" to refer to the task of estimating an audio signal's f_0.

Given the importance of accurate pitch estimation, the topic has received a considerable amount of research attention over the past decades. Numerous digital signal processing (DSP) techniques estimate pitch based on the cepstrum [1], the power spectrum [2–4], or the autocorrelation function [5, 6].

More recently, deep neural networks have been applied to the task of pitch estimation [7–10]. In a typical architecture, a convolutional neural network (CNN) is given overlapping frames of raw audio as input and trained to predict a probability distribution over a discrete set of f_0 candidates in a supervised fashion. While these models can reach very high accuracy, they require a large amount of training data annotated with reliable ground truth pitch values and can struggle with out-of-domain generalization and robustness to noise and reverberation. Additionally, the CNNs usually consist of millions of parameters, making them less suitable for use in low-resource environments.

These drawbacks have been addressed in two different ways. Self-supervised training paradigms [11–14] do not require labeled training data, and incorporating traditional DSP approaches into neural networks has been shown to increase efficiency and robustness [15, 16].

In this paper, we combine these two approaches and merge task-specific DSP-based features with both supervised and self-supervised training paradigms. Specifically, we substitute the audio frontend in the *Pitch Estimation with Self-Supervised Transposition-Equivariant Objective* (PESTO) architecture [14] for a representation obtained from the *Sawtooth Waveform Inspired Pitch Estimator* (SWIPE) [3], which estimates pitch by measuring the similarity of the input spectrum to that of sawtooth waves at various pitch candidates. We also investigate the use of SWIPE as a frontend for supervised neural pitch estimators, which usually operate directly on the audio waveform.

The core insights of our work are these:

- Although SWIPE is commonly used as a baseline for neural pitch estimation, we find that its performance has been significantly underreported. We show that SWIPE in its original form surpasses the accuracy of the state of the art in self-supervised neural pitch detection (PESTO).
- SWIPE is a well-suited audio frontend for neural pitch estimators in both supervised and self-supervised settings, and can improve the state-of-the-art in terms of accuracy, robustness, efficiency, and latency.

Trained models alongside a SWIPE implementation in PyTorch [17] are available online. [1] The remainder of this paper is structured as follows. In Section 2, we give an overview over SWIPE and neural pitch estimation methods. Section 3 covers some aspects of our SWIPE implementation choices and details how we embed SWIPE into neural pitch estimation architectures. Section 4 describes how we evaluate our approach, and Section 5 presents the results of our evaluation.

[1] https://github.com/dsuedholt/neural-pitch-swipe

© D. Marttila and J. D. Reiss. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** D. Marttila and J. D. Reiss, "Improving Neural Pitch Estimation with SWIPE Kernels", in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.,* Daejeon, South Korea, 2025.

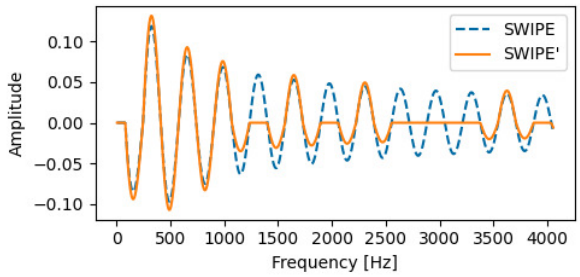


Figure 1. SWIPE and SWIPE' kernels corresponding to a pitch candidate at 330 Hz. The SWIPE kernel contains peaks at all integer harmonics of the candidate frequency. The SWIPE' kernel is obtained by removing the peaks at non-prime harmonics.

2. BACKGROUND

2.1 SWIPE

The *Sawtooth Waveform Inspired Pitch Estimator* (SWIPE) [3] estimates pitch by identifying the fundamental frequency of a sawtooth waveform whose spectrum best matches that of the input signal. To achieve this, it constructs spectral kernels for a number of discrete pitch candidates, and assigns a score to each pitch candidate by measuring the similarity between its associated kernel and the spectrum of the input signal.

2.1.1 Score Calculation

More formally, consider a (windowed) audio signal $x[n]$ of length N and its discrete Fourier Transform (DFT) $X[k]$, which may be truncated to the $K = \lfloor N/2 \rfloor + 1$ bins corresponding to non-negative frequencies if x is real-valued. Let now $C = \{f_1, f_2, \dots f_{|C|}\}$ be a set of $|C|$ pitch candidates. Then $S_c[k]$ is the spectral kernel associated with the pitch candidate f_c, and we can compute its SWIPE score $Z(f_c) : C \to [-1, 1]$ as the normalized inner product between S_c and X:

$$Z(f_c) = \frac{\sum_{k=0}^{K-1} S_c[k] \cdot |X[k]|^{1/2}}{\left(\sum_{k=0}^{K-1} |X[k]|\right)^{1/2}} \tag{1}$$

The pitch estimate is then given by the f_c that maximizes $Z(f_c)$ and may optionally be further refined by e.g. parabolic interpolation of the local maximum.

While Eqn (1) computes the inner product over all bins of the DFT for notational simplicity, the original SWIPE paper suggests resampling the spectrum to the Equivalent Rectangular Bandwidth (ERB) [18] scale for speech data, or to the mel scale for musical instruments.

2.1.2 Kernel Design

The kernel S_c is designed to maximize the inner product with X if x is a signal with fundamental frequency $f_0 = f_c$. To achieve that, it contains cosine lobes of width $f_c/2$ at all integer harmonics of f_c, decaying in magnitude to mimic the spectrum of a sawtooth wave. As the authors of SWIPE lay out, this corresponds precisely to the square root of the main lobes of a Hann-windowed sawtooth wave if the size of the analysis window is exactly $T = 8/f_c$. The kernel further contains negative-valued valleys at $\frac{1}{2}f_c, \frac{3}{2}f_c, \dots$, i.e. at the midpoint between each harmonic peak.

Since all harmonics of a signal with a true fundamental frequency of f also contribute to the scores of the pitch candidates at $f/2, f/3, \dots$, a common variant of the SWIPE algorithm removes the non-prime harmonics (except for the first one) of all kernels to reduce the problem of octave errors. This is known as SWIPE', but effective and widespread enough that it is often simply referred to as SWIPE, for example in the Speech Processing Toolkit (SPTK) [19] implementation, which is based directly on the MATLAB code published along with SWIPE. We take the same approach in this paper and will generally assume that the scores Z are calculated using SWIPE' kernels. An example of such a kernel is illustrated in Figure 1.

2.2 Supervised Neural Pitch Estimation

The established way of using neural networks for pitch estimation is to interpret an audio signal $x[n]$ as a vector $\mathbf{x}$. A network f_θ then maps $\mathbf{x}$ to a vector $\mathbf{y} \in [0, 1]^{|C|}$, where each entry y_c represents the probability that a corresponding pitch candidate f_c is the pitch of $\mathbf{x}$. In supervised training, this is treated as a multi-class classification problem, calculating the loss using the cross-entropy to the ground truth pitch. The ground truth distribution may be smoothed using Gaussian blurring to aid training [7]. Voicing confidence can be deduced from the entropy of the predicted probability distribution [9].

2.3 Self-Supervised Neural Pitch Estimation

Pitch Estimation with Self-Supervised Transposition-Equivariant Objective (PESTO) [14] is a state-of-the-art architecture for self-supervised training of neural network pitch estimators, where natural symmetries of the input are exploited to learn a translation-equivariant representation, instead of providing ground truth pitches to the model.

2.3.1 Training Setup

During training, the model learns to optimize a combination of three losses:

An **equivariance loss** enforces that pitch-shifted versions of an input should result in output distributions that are transpositions of the original input. Given an input $\mathbf{x}$ and its pitch-shifted version $\mathbf{x}^{(k)}$ (shifted by k semitones), their respective outputs $\mathbf{y}$ and $\mathbf{y}^{(k)}$ should satisfy $\phi(\mathbf{y}^{(k)}) = \alpha^k \phi(\mathbf{y})$, where ϕ is a deterministic linear mapping:

$$\begin{array}{rccc} \phi & : & \mathbb{R}^{|C|} & \to & \mathbb{R} \\ & & \mathbf{y} & \mapsto & (\alpha, \alpha^2, \dots, \alpha^{|C|})\mathbf{y} \end{array} \tag{2}$$

and α is a hyperparameter.

A **regularization loss** further ensures that the network's outputs for pitch-shifted inputs maintain the expected transposition relationship. For a pair of outputs $\mathbf{y}$

and $\mathbf{y}^{(k)}$, the shifted cross-entropy loss

$$\mathcal{L}_{\text{SCE}}(\mathbf{y}, \mathbf{y}^{(k)}, k) = \sum_{i=0}^{|C|-1} y_i \log\left(y_{i+k}^{(k)}\right) \tag{3}$$

measures how well $\mathbf{y}^{(k)}$ matches the k-semitone shift of $\mathbf{y}$.

Finally, an **invariance loss** encourages the mapping f_θ to be invariant to the timbre of the signal. During training, PESTO draws random transforms t from a set of pitch-preserving data augmentations $\mathcal{T}$. Given $\tilde{\mathbf{x}} = t(\mathbf{x})$, the invariance loss is then expressed as the cross-entropy between $\mathbf{y} = f_\theta(\mathbf{x})$ and $\tilde{\mathbf{y}} = f_\theta(\tilde{\mathbf{x}})$.

2.3.2 Model Architecture

The PESTO architecture uses the constant-Q transform (CQT) of an audio frame as its input, where the bins of the CQT exactly correspond to the pitch candidates. The transforms $\mathcal{T}$ take the form of adding random noise and gain to the CQT frames. A CNN processes the frame, and its flattened output is fed to a final linear layer followed by a softmax layer which produces a probability distribution. Importantly, the final linear layer uses a Toeplitz matrix as its weight matrix to preserve the transposition equivariance of the CNN.

3. METHODS

The core insight of this work is that SWIPE scores encode rich pitch information and are thus well suited as an audio frontend for neural pitch estimation in both supervised and self-supervised settings. This section first covers our implementation of SWIPE in detail, and then describes how we adapt supervised and self-supervised neural pitch estimators to work with SWIPE scores.

3.1 SWIPE Implementation

We calculate the SWIPE scores by sampling the spectrum at 1024 frequencies, which are linearly spaced on the mel scale over a range from $0.25 \cdot f_{\min}$ to $1.25 \cdot f_{\max}$. We used the Slaney-style mel scale, which is linear up to 1 kHz and logarithmic above, as implemented in the librosa toolkit [20]. For each of the various window sizes, the spectrum is calculated with the same FFT resolution (zero-padding the input as needed) and evaluated at the sampling frequencies using linear interpolation. We arrange the pitch candidates to match the CQT resolution used in PESTO: logarithmically spaced over a range of $f_{\min} = 27.5$ Hz to $f_{\max} = 8055$ Hz, using a resolution of 3 bins per semitone, for a total of 295 bins.

Although many papers on neural pitch estimation compare their work to SWIPE as a baseline, they generally do not cite the implementation they used or report the parameters they chose. To make sure that our implementation does not significantly underperform, we compare it to the most popular open-source implementation of SWIPE, which is contained in the Speech Processing Toolkit (SPTK) [19]. It uses 8 pitch bins per semitone by default, samples the input frequency spectrum according to the ERB [18] scale, and refines the estimate using parabolic interpolation.

Table 1 contains the Raw Pitch Accuracy (RPA) achieved by the SPTK and our implementation on the MDB-stem-synth and MIR-1K datasets and compares it to previously reported baseline values. The metrics and datasets are described in more detail in Section 4. The accuracy of the SPTK implementation seems to significantly deteriorate for large search ranges. We report the values for upper limits of 2 kHz and 8 kHz, where the lower limit is 30 Hz for both. We set the score threshold which pitch candidates need to exceed to be considered to 0.

Our implementation appears to be a lot more robust to its large search range (27.5–8055 Hz). Switching from ERB to mel sampling results in a notable accuracy gain on MDB-stem-synth, which contains more varied timbres. This matches the results of the original SWIPE paper.

Both the SPTK and our own implementation can perform much more accurately than the values that were previously reported as baselines in the neural pitch estimation literature suggest, to the extent that SWIPE outperforms even state-of-the-art self-supervised pitch detection models (see Section 5.2).

3.2 Supervised Neural Pitch Estimation

We experiment with using both SWIPE scores and the CQT as an audio frontend in a supervised training context.

We feed the input into a CNN with 6 1D-convolutional layers, applying layer normalization and a leaky ReLU non-linearity with slope 0.3 between each layer. Zero-padding is applied to the input in each layer to preserve the input dimension. After flattening, the output of the final layer is reduced to the dimensionality of the pitch bins and fed into a Softmax layer to obtain a probability distribution. We find that using a dense linear layer to perform the dimensionality reduction strongly degraded generalization in this setup, and instead also employ a Toeplitz layer in the supervised model.

3.3 Self-Supervised Neural Pitch Estimation

The original PESTO architecture is already well suited to work with SWIPE scores, which can be directly substituted for the CQT bins without violating the assumptions on translation equivariance. Since SWIPE scores encode periodicity information much more explicitly than CQT frames, we expect the encoder network to achieve similar performance with fewer parameters. We test this hypothesis by training a PESTO-style encoder with a drastically reduced parameter count, consisting only of the final Toeplitz fully-connected layer – a convolutional layer with a single filter of size 647 – and softmax normalization. In this very simple architecture, the Toeplitz layer can be seen as essentially learning a reweighting of the SWIPE scores, and to refine the location of the peak score if the output resolution is larger than the input resolution.

Initial experiments indicated that applying random data augmentation to the SWIPE scores only resulted in degraded performance compared to the baseline DSP algo-

	Reported in [12, 13]	SPTK (2 kHz)	SPTK (8 kHz)	Ours (ERB)	Ours (mel)
MIR-1K	86.6%	96.5%	68.2%	95.7%	96.2%
MDB	90.7%	94.1%	61.4%	94.0%	96.1%

Table 1. Raw Pitch Accuracy obtained by SWIPE implementations on the MIR-1K and MDB-stem-synth datasets, compared to baseline values previously reported in self-supervised pitch estimation papers. For SPTK, we report different upper limits for the search range. For our implementation, the search range is constant, but the frequency sampling scale changes.

rithm. We additionally augment the audio frame in the time domain by adding random noise and applying a finite impulse response (FIR) filter with a randomized amplitude response. This results in an increased computational cost for training, since the SWIPE scores need to be recalculated at every training step, but does not affect the computational cost of inference once training has finished.

4. EXPERIMENTAL SETUP

4.1 Datasets

Our experiments use three f_0-annotated datasets that are commonly used for training and benchmarking pitch detectors:

MDB-stem-synth [21] contains 230 solo tracks (418 minutes total) of instrument sounds and vocals. The audio is re-synthesized from its f_0 annotations, which means that the f_0 annotations are perfect. It is annotated with a hop size of 2.9 ms.

PTDB-TUG [22] contains 4720 audio and laryngograph recordings (576 minutes total) of 20 English speakers reading sentences. It is annotated with a hop size of 10 ms.

MIR-1K [23] contains 1000 short recordings (133 minutes total) of Chinese karaoke performances. It is annotated with a hop size of 20 ms.

4.2 Baselines

We compare all results to two DSP-based pitch detection baselines: PYIN [24] and SWIPE. We do not perform Viterbi decoding or any sort of peak refinement, simply selecting the pitch candidate with the highest score. PYIN scores are based on autocorrelation and so their natural resolution is expressed in integer samples. We resample the scores to the same pitch candidate resolution as used for SWIPE using linear interpolation.

As a baseline for supervised training, we choose FCNF0++ [9], which to the best of our knowledge is the currently best-performing supervised monophonic neural pitch detector that operates on a frame-by-frame basis, rather than processing the entire audio signal at once.

In the self-supervised setting, we compare our results against the original PESTO architecture, which is the current state of the art in self-supervised monophonic neural pitch estimation.

4.3 Evaluation Metrics

We use the mir_eval package [25] to report the following metrics:

Raw Pitch Accuracy (RPA), the percentage of voiced frames for which the model predicted a pitch within 50 cents of the ground truth.

F-Score, measuring the accuracy of the binary voiced/unvoiced decision.

Overall Accuracy (OA), the percentage of all frames (voiced and unvoiced) for which a correct voicing decision was made, and for which the model predicted a pitch within 50 cents of the ground truth if the frame is voiced.

5. RESULTS AND DISCUSSION

We report separate experimental results for the supervised and self-supervised approaches, in each case closely replicating the training setup of the baselines (FCNF0++ and PESTO, respectively) to assess the impact of using SWIPE scores as an audio frontend.

5.1 Supervised Models

We refer to the two proposed supervised models (see Section 3.2) as **CQT-sup** and **SWIPE-sup**. We train our models as well as the FCNF0++ baseline on MDB-stem-synth and PTDB-TUG at the same time. While networks that take CQT or SWIPE scores as input are sample-rate agnostic, FCFN0++ operates on the raw audio waveform and was designed to work with a sampling rate of 8 kHz, so we resample its input accordingly.

In the interest of a direct comparison, we use the 70-15-15 split into training, validation and testing partitions that was published in [9]. The performance of the trained models is measured by calculating RPA, F-Score and OA on the testing set. To better measure generalization performance on unseen data, we additionally evaluate the trained models on the full MIR-1K dataset, which is not used in training.

We train the models for 500,000 steps, using a batch size of 256 and the Adam optimizer [26] with an initial learning rate of 0.0002. Table 2 shows the results of the evaluation. Overall, both CQT-sup and SWIPE-sup seem competitive with FCNF0++, but not clearly superior. They are able to almost match the in-domain RPA of FCNF0++, and outperform it in terms of voiced/unvoiced accuracy and generalization. The two proposed models use fewer trainable parameters than FCNF0++, but require a larger context window.

The CQT input features seem to be particularly well suited for making voiced/unvoiced decisions, with the CQT-sup model attaining the highest F-Score on both the test set and on MIR-1K.

Method	# Params	Window Size [ms]	Test Partition			MIR-1K		
			RPA	F-Score	OA	RPA	F-Score	OA
PYIN	-	145	89.4%	-	-	95.4%	-	-
SWIPE	-	327	93.2%	-	-	**96.2%**	-	-
CQT-sup (ours)	0.9M	1871	98.1%	**98.7%**	**99.2%**	92.2%	**91.2%**	87.1%
SWIPE-sup (ours)	0.9M	327	97.9%	98.1%	98.6%	93.5%	90.0%	**87.8%**
FCNF0++ [9]	6.6M	128	**98.3%**	98.2%	98.7%	91.0%	87.8%	86.0%

Table 2. Evaluation results of the supervised models as measured by Raw Pitch Accuracy (RPA), the F-Score for the voiced/unvoiced decision, and Overall Accuracy (OA). We also report the sizes of the neural networks and the maximum window size required by the estimator. Models are evaluated using the combined test partition of MDB-stem-synth and PTDB-TUG published in [9], as well as on the entire MIR-1K dataset, which was not used in training.

All three models struggle with generalization, staying well behind the DSP baselines on MIR-1K. The best generalization behavior is shown by SWIPE-sup, even though it was the least accurate model on the test set.

5.2 Self-Supervised Models

We refer to the three modified PESTO models (see Section 3.3) as **CQT-tiny**, **SWIPE-full**, and **SWIPE-tiny**, where "tiny" refers to the Toeplitz-only encoder and "full" to the original PESTO encoder architecture with a multi-layer CNN. We train the models on the whole of MIR-1K and measure their performance on MDB-stem-synth, and vice versa. The models are trained for 50 epochs using a batch size of 256 and the Adam optimizer with an initial learning rate of 0.0001.

The results of the evaluation are given in Table 3. The baseline SWIPE implementation outperforms PESTO on both datasets, regardless of which dataset the model was trained on. **This means that the original SWIPE algorithm outperforms all work on self-supervised monophonic pitch detection published to date.**

When using CQT frames as input, reducing the encoder network to just the final Toeplitz layer noticeably degrades performance, especially in the across-dataset evaluation. However, it is worth noting that CQT-tiny still achieves relatively good same-dataset accuracy. Since no explicit pitch information is given to the model during training, this is a strong indicator for the usefulness of the transposition-equivariant training structure that PESTO introduced.

The highest accuracy on the same-dataset evaluation is achieved by the two models that use SWIPE scores as input. Like CQT-tiny however, their performance plummets when trained on MIR-1K and evaluated on MDB-stem-synth. MDB-stem-synth covers a larger pitch range than MIR-1K and contains more varied timbres, making generalization challenging. The original PESTO is the only model that is able to make this jump reasonably well.

In the reverse direction however, SWIPE-tiny achieves the highest RPA out of the four models when trained on MDB-stem-synth and evaluated on MIR-1K, as well as the best same-dataset RPA for MDB-stem-synth. Adding the Toeplitz layer on top of the SWIPE scores improves their performance, but the additional network layers in SWIPE-full do not bring further accuracy gains, seemingly hindering performance instead.

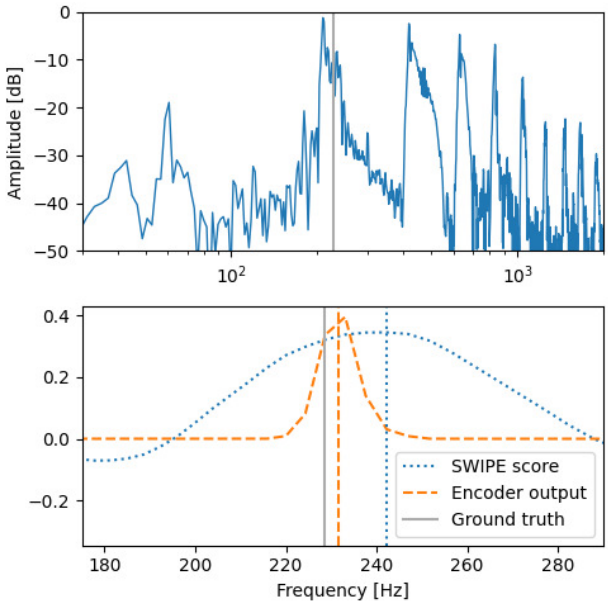


Figure 2. **Top:** The spectrum of a frame of audio from MIR-1K. The solid vertical line marks the ground truth pitch. **Bottom:** The SWIPE scores for the frame, before (dotted) and after (dashed) they were transformed by the SWIPE-tiny encoder trained on MDB-stem-synth. The vertical lines indicate the pitch estimate obtained from the basic SWIPE algorithm (dotted), the estimate given by SWIPE-tiny (dashed), and the ground truth (solid).

The Toeplitz-only encoder in SWIPE-tiny seems to learn to refine the peaks of the SWIPE scores, mitigating errors caused by the quantization of the search space or by input spectra that deviate too far from the harmonic ideal. Figure 2 illustrates a frame where an estimation error of 90 cents is reduced to 30 cents after feeding the SWIPE scores through the SWIPE-tiny encoder.

5.3 Latency-Accuracy Tradeoff for SWIPE

The frame-based structure of the evaluated models, and especially the lightweight architecture of the self-supervised estimators, lend themselves naturally to use in real-time, streaming applications. In this context, small window sizes are desirable to reduce latency.

As described in Section 2.1, the theoretical ideal win-

Method	# params	Trained on	Raw Pitch Accuracy MIR-1K	MDB-stem-synth
PYIN	-	-	95.4%	91.6%
SWIPE	-	-	96.2%	96.1%
PESTO (baseline from [14])	28.9k	MIR-1K	96.1%	**94.6%**
		MDB-stem-synth	93.5%	95.5%
CQT-tiny	647	MIR-1K	95.6%	78.8%
		MDB-stem-synth	91.7%	95.5%
SWIPE-full	28.2k	MIR-1K	**97.0%**	89.7%
		MDB-stem-synth	96.1%	96.4%
SWIPE-tiny	647	MIR-1K	96.6%	90.1%
		MDB-stem-synth	**96.4%**	**96.5%**

Table 3. Evaluation results of the self-supervised models. For both datasets, we highlight the best result achieved when training and evaluating on the same dataset (no explicit pitch information is provided to the model during training), and when training on one dataset and evaluating on the other. The performance of PYIN and SWIPE is given for comparison.

Window Size [Samples]	[ms]	Raw Pitch Accuracy SWIPE-tiny	SWIPE-sup
16384	372	96.4%	97.2%
8192	186	96.4%	97.1%
4096	93	96.2%	96.7%
2048	46	85.0%	86.9%

Table 4. The effect of reducing the maximum window size (for a sampling rate of 44.1 kHz) at which SWIPE scores are calculated. RPA on MIR-1K is reported for SWIPE-tiny (trained on MDB-stem-synth) and SWIPE-sup.

dow size for each pitch candidate f_c is exactly $8/f_c$. In practice however, it is sufficient to only consider window sizes whose length in samples is a power of two. For a given pitch candidate with an ideal window length W, the score is then calculated twice at window lengths $2^{\lfloor \log_2(W) \rfloor}$ and $2^{\lceil \log_2(W) \rceil}$, and linearly interpolated to obtain an approximation to the score at the ideal size. Given a sampling rate of $f_s = 44.1$ kHz and a minimum pitch candidate of $f_{\min} = 27.5$ Hz, the next-longest window with a power-of-two length in samples corresponds to 327 ms, which is already a significant improvement compared to the 1871 ms required by the CQT.

However, this can be reduced further. The SWIPE-based models offer a straightforward way to reduce both latency and computational cost at the expense of accuracy by simply calculating the scores for lower pitch candidates at shorter window sizes (without interpolation). Crucially, this adjustment can be made flexibly at inference time without retraining the model. Table 4 shows the effect that reducing the window size has on the RPA for two selected models. Note that reducing the window size to a length that is not a power of two is also possible if finer control over the tradeoff is desired.

5.4 Robustness to Noise

In the final experiment, we investigate how robust various trained models are to noisy conditions by adding white noise to the input audio at decreasing signal-to-noise ratios.

Model	Raw Pitch Accuracy (MIR-1K) clean	5 dB	0 dB	-5 dB	-10 dB
PYIN	95.4%	95.3%	95.1%	**93.7%**	85.8%
SWIPE	96.2%	93.9%	91.2%	85.6%	75.2%
CQT-sup	92.2%	91.5%	89.3%	87.3%	82.3%
SWIPE-sup	93.5%	91.6%	90.0%	87.1%	72.2%
SWIPE-tiny	**96.6%**	**96.0%**	**95.3%**	93.4%	**88.5%**
PESTO	94.6%	93.3%	92.9%	90.1%	81.7%
FCNF0++	91.0%	90.3%	89.0%	83.5%	81.0%

Table 5. The effect that adding white noise at various signal-to-noise ratios to the input audio has on the raw pitch accuracy on the MIR-1K dataset for various models.

The results are shown in Table 5. SWIPE-tiny appears to be fairly robust to background noise, especially compared to the base SWIPE algorithm. The performance of the supervised models degrades somewhat quicker than that of the self-supervised ones. This is not too surprising, since invariance to added noise is an explicit training objective for the self-supervised models (see Section 3.3).

6. CONCLUSION

We investigated the potential of combining the SWIPE algorithm with neural pitch estimation. We adapted established supervised and self-supervised training techniques to use SWIPE scores as an audio frontend and obtained accurate, efficient, robust and flexible pitch estimators.

We demonstrated that the potential of SWIPE has been significantly underestimated in the literature despite being commonly used as a baseline. The algorithm in its original form outperforms state-of-the-art self-supervised neural pitch estimators.

In the future, we plan to explore whether the performance of the pitch estimators can be further improved by hybrid training schemes that make simultaneous use of labeled data and self-supervised training objectives. We would also like to investigate whether certain parameters of SWIPE, such as the frequency envelope or the weights of individual harmonics, can be directly learned from data.

7. ACKNOWLEDGMENTS

This work was supported by UK Research and Innovation [grant number EP/S022694/1]. The authors would like to thank the anonymous reviewers for their valuable feedback which significantly improved this paper.

8. REFERENCES

[1] A. M. Noll, "Cepstrum Pitch Determination," *The Journal of the Acoustical Society of America*, vol. 41, no. 2, pp. 293–309, Feb. 1967.

[2] R. C. Maher and J. W. Beauchamp, "Fundamental frequency estimation of musical signals using a two-way mismatch procedure," *The Journal of the Acoustical Society of America*, vol. 95, no. 4, pp. 2254–2263, 1994.

[3] A. Camacho and J. G. Harris, "A sawtooth waveform inspired pitch estimator for speech and music," *The Journal of the Acoustical Society of America*, vol. 124, no. 3, pp. 1638–1652, Sep. 2008.

[4] S. Gonzalez and M. Brookes, "PEFAC - A Pitch Estimation Algorithm Robust to High Levels of Noise," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 2, pp. 518–530, Feb. 2014.

[5] A. de Cheveigné and H. Kawahara, "YIN, a Fundamental Frequency Estimator for Speech and Music," *The Journal of the Acoustical Society of America*, vol. 111, no. 4, pp. 1917–1930, 2002.

[6] P. McLeod and G. Wyvill, "A smarter way to find pitch," in *International Computer Music Conference (ICMC)*, 2005.

[7] J. W. Kim, J. Salamon, P. Li, and J. P. Bello, "CREPE: A Convolutional Representation for Pitch Estimation," in *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2018.

[8] L. Ardaillon and A. Roebel, "Fully-Convolutional Network for Pitch Estimation of Speech Signals," in *Interspeech 2019*, 2019.

[9] M. Morrison, C. Hsieh, N. Pruyne, and B. Pardo, "Cross-domain Neural Pitch and Periodicity Estimation," http://arxiv.org/abs/2301.12258, Jun. 2023.

[10] X. Li, H. Huang, Y. Hu, L. He, J. Zhang, and Y. Wang, "YOLOPitch: A Time-Frequency Dual-Branch YOLO Model for Pitch Estimation," in *Interspeech 2024*, 2024.

[11] B. Gfeller, C. Frank, D. Roblek, M. Sharifi, M. Tagliasacchi, and M. Velimirović, "Pitch estimation via self-supervision," in *2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2020.

[12] B. Gfeller, C. Frank, D. Roblek, M. Sharifi, M. Tagliasacchi, and M. Velimirovic, "SPICE: Self-Supervised Pitch Estimation," *IEEE/ACM Transactions on Audio Speech and Language Processing*, vol. 28, pp. 1118–1128, 2020.

[13] J. Engel, R. Swavely, A. Roberts, L. H. Hantrakul, and C. Hawthorne, "Self-Supervised Pitch Detection by Inverse Audio Synthesis," *Workshop on Self-Supervision in Audio and Speech at the 37th International Conference on Machine Learning (ICML 2020)*, 2020.

[14] A. Riou, S. Lattner, G. Hadjeres, and G. Peeters, "PESTO: Pitch estimation with self-supervised transposition-equivariant objective," in *Proceedings of the 24th International Society for Music Information Retrieval Conference (ISMIR)*, 2023.

[15] P. Rengaswamy, M. G. Reddy, K. S. Rao, and P. Dasgupta, "hf0: A Hybrid Pitch Extraction Method for Multimodal Voice," *Circuits, Systems, and Signal Processing*, vol. 40, no. 1, pp. 262–275, Jan. 2021.

[16] E. S. Hassan, B. Neyazi, H. S. Seddeq, A. Z. Mahmoud, A. S. Oshaba, A. El-Emary, and F. E. Abd El-Samie, "HAEPF: Hybrid approach for estimating pitch frequency in the presence of reverberation," *Multimedia Tools and Applications*, vol. 83, no. 32, pp. 77 489–77 508, Feb. 2024.

[17] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.

[18] B. R. Glasberg and B. C. Moore, "Derivation of auditory filter shapes from notched-noise data," *Hearing Research*, vol. 47, no. 1, pp. 103–138, 1990.

[19] T. Yoshimura, T. Fujimoto, K. Oura, and K. Tokuda, "SPTK4: An open-source software toolkit for speech signal processing," in *12th ISCA Speech Synthesis Workshop (SSW 2023)*, 2023, pp. 211–217.

[20] B. McFee, C. Raffel, D. Liang, D. P. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and music signal analysis in python." in *Proceedings of the 14th Python in Science Conference*, 2015.

[21] J. Salamon, R. M. Bittner, J. Bonada, J. J. Bosch, E. Gómez, and J. P. Bello, "An analysis/synthesis framework for automatic f0 annotation of multitrack datasets." in *Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR)*, 2017.

[22] G. Pirker, M. Wohlmayr, S. Petrik, and F. Pernkopf, "A pitch tracking corpus with evaluation on multipitch tracking scenario." in *Interspeech*, 2011, pp. 1509–1512.

[23] C.-L. Hsu and J.-S. R. Jang, "On the improvement of singing voice separation for monaural recordings using the mir-1k dataset," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 18, no. 2, pp. 310–319, 2010.

[24] M. Mauch and S. Dixon, "pYIN: A fundamental frequency estimator using probabilistic threshold distributions," in *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2014, pp. 659–663.

[25] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, D. P. Ellis, and C. C. Raffel, "Mir_eval: A transparent implementation of common mir metrics." in *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, 2014.

[26] D. P. Kingma and J. L. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations (ICLR)*, 2015.